Earth-2-Earth Reusable code

MongoDB Queries -

- /collections: Each collection is a device (sensor station)

```
const database = mongoclient.db(databaseName);
const collectionsList = await database.listCollections().toArray();
const collectionNames = collectionsList.map(col => col.name);
```

- Earliest and latest timestamp (`/date-range`) Earliest and latest dates with data. So user doesn't have to guess how far back they can see

```
const [doc] = await col.aggregate([
  {
    $project: {
      t: {
        $dateFromString: {
          dateString: "$Timestamp.time_local",
          onError: null,
          onNull: null,
        },
      },
    },
  },
  { $match: { t: { $ne: null } } },
  { $group: { _id: null, minDate: { $min: "$t" }, maxDate: { $max: "$t" } } },
]).toArray();

// doc → { minDate, maxDate }, or undefined if the collection is empty
```

- Packets in a time range (`/data?startTime=&endTime=`): User can select a time range and data from that time period will be retrieved

```
const start = new Date(startTime);
const end = new Date(endTime);

const packets = (
  await collection.aggregate([
    {
      $addFields: {
        _t: {
```

```
          $dateFromString: {
            dateString: "$Timestamp.time_local",
            onError: null,
            onNull: null,
          },
        },
      },
    },
    { $match: { _t: { $gte: start, $lt: end } } },
    { $sort: { _t: -1 } },
    { $project: { _t: 0, Packet: 0, WiFi: 0 } },
  ]).toArray()
).reverse();
```

- Last X packets (`/data?x=`) User can say show me the last "X" packets

```
const packets = (
  await collection
    .find({}, { projection: { Packet: 0 } })
    .sort({ "Timestamp.time_local": -1 })
    .limit(x)
    .toArray()
).reverse();
```

- Deployment metadata (`/metadata`): Information about a deployment (When a station was put it, ,Where it is physically located longitude/latitude, who runs the station)

```
const collections = await database.listCollections().toArray();
let packet = null;

for (const c of collections) {
 packet = await database.collection(c.name).findOne({ type: "metadata" });
 if (packet) break;
}
```

1. A. Data retrieval — High reuse potential server.js — /data, /databases, /collections, Ear2Earth already uses Express and the MongoDB driver to retrieve sensor readings, including recent packets and custom time ranges. Farm2Table can adapt these queries to retrieve measurements when a user selects a station on the map.
2. B. Time-series plots — High reuse potential index.js Ear2Earth already creates Plotly charts with zooming, panning, and measurement inspection. It also includes synchronized x-axis behavior in syncThisPlot(). The charting logic could be extracted into a reusable Farm2Table chart component, separate from the existing sound controls.

3.
4. C. Date-range selection — High reuse potential server.js — /date-range index.js - setDateBoundsForSelection() Ear2Earth can retrieve the earliest and latest available readings for a sensor dataset and supports custom date selection. Farm2Table can extend this with presets for 24 hours, 7 days, 30 days, and growing seasons.
5. D. CSV export — High reuse potential index.js — generateCSV() It already produces CSV data from plotted measurements. Farm2Table can adapt this to export selected stations, variables, and time ranges, with clearer unit and timezone handling. **will have to be modified since this take the points from the plotly chart, we might one to get it straight from MongoDB**